

BDR – ATIVIDADE 12 – GABRIEL MOURA

EX01.A)

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'bdr_aula12'. The 'Query' tab is active, displaying the following SQL query:

```
25 FROM 'C:/caminho/aula3_carro.csv'
26 DELIMITER ','
27 CSV HEADER;
28
29 -- A
30
31 EXPLAIN ANALYZE
32 SELECT *
33 FROM pessoa
34 WHERE nome = 'Ana Silva';
```

The 'Data Output' tab shows the query plan:

Step	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=10.020..33.617 rows=5.00 loop=1)
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=10.020..33.617 rows=5.00 loop=1)
2	Filter: ((nome)::text = 'Ana Silva':text)
3	Rows Removed by Filter: 99995
4	Buffers: shared hit=688
5	Planning:
6	Buffers: shared hit=31 dirtied=1
7	Planning Time: 11.040 ms
8	Execution Time: 36.511 ms

Total rows: 8 Query complete 00:00:00.146

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to 'bdr_aula12'. The 'Query' tab is active, displaying the following SQL query:

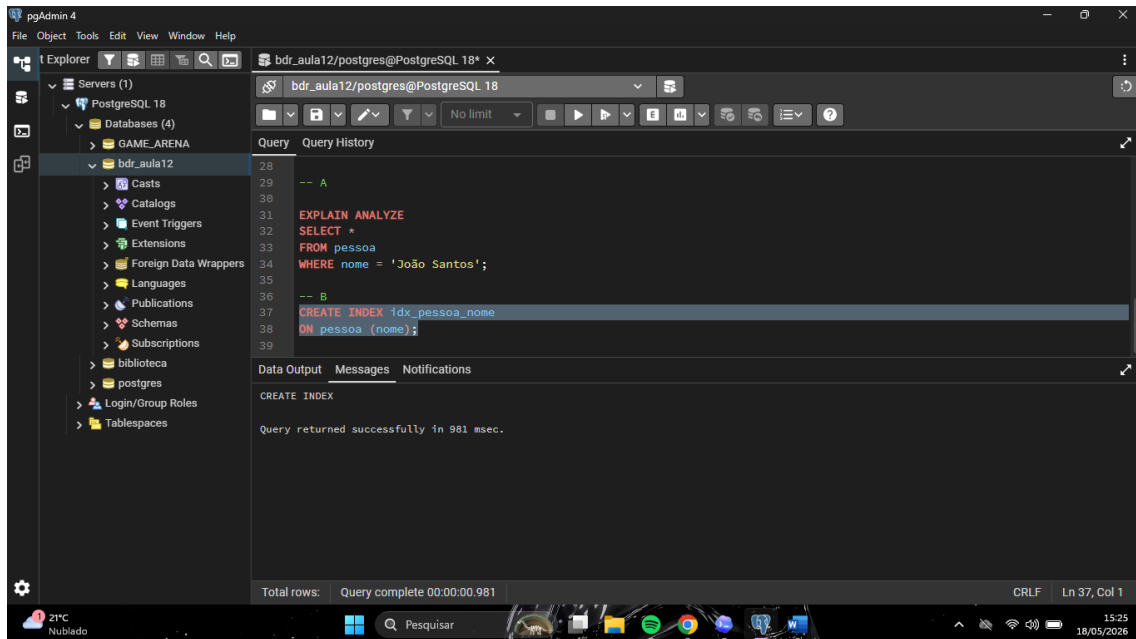
```
23
24 COPY carro (id_carro, placa, ano, id_pessoa)
25 FROM 'C:/caminho/aula3_carro.csv'
26 DELIMITER ','
27 CSV HEADER;
28
29 -- A
30
31 EXPLAIN ANALYZE
32 SELECT *
33 FROM pessoa
34 WHERE nome = 'João Santos';
```

The 'Data Output' tab shows the query plan:

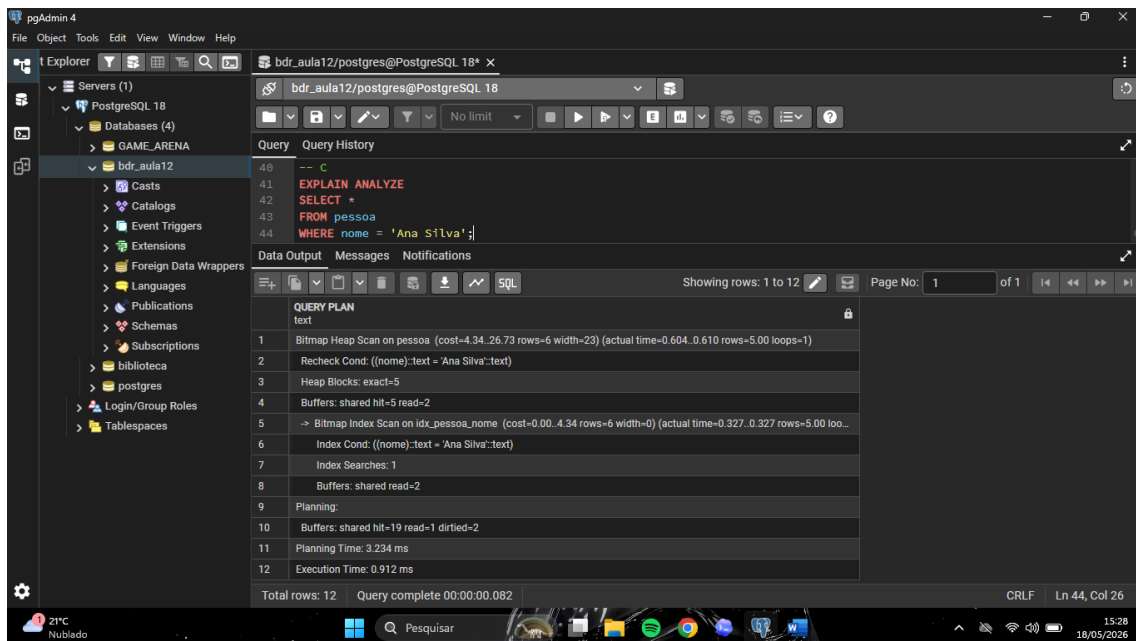
Step	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=3.151..14.602 rows=6.00 loop=1)
1	Seq Scan on pessoa (cost=0.00..1938.00 rows=6 width=23) (actual time=3.151..14.602 rows=6.00 loop=1)
2	Filter: ((nome)::text = 'João Santos':text)
3	Rows Removed by Filter: 99994
4	Buffers: shared hit=688
5	Planning Time: 0.326 ms
6	Execution Time: 14.664 ms

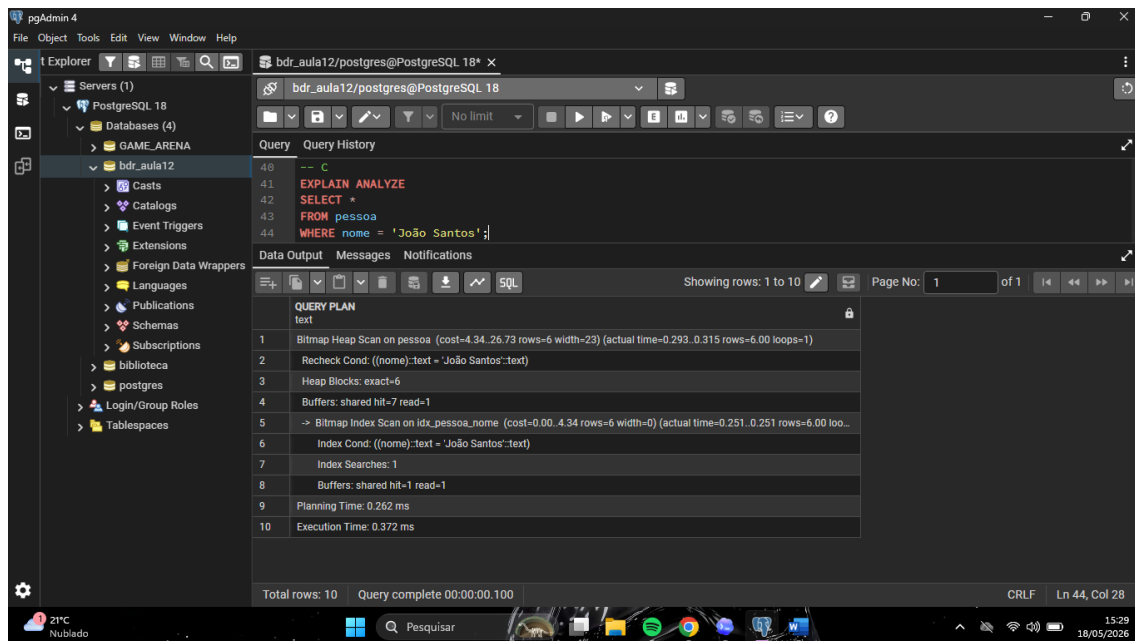
Total rows: 6 Query complete 00:00:00.131

B)



C)





D)

- Qual plano foi utilizado antes do índice?

Antes da criação do índice, o PostgreSQL usou **Seq Scan on pessoa** nas duas consultas (Ana Silva e João Santos)

- Qual plano foi utilizado após o índice?

Depois de criar `idx_pessoa_nome`, o plano passou a usar o índice com:

Bitmap Index Scan on idx_pessoa_nome + Bitmap Heap Scan on pessoa

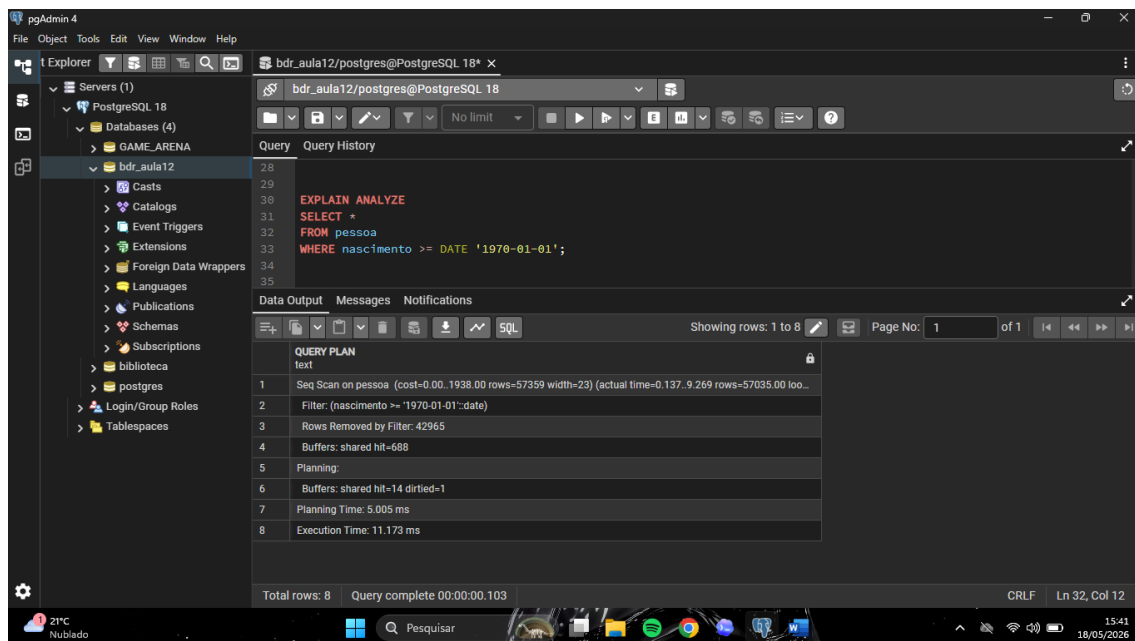
- Houve redução no tempo de execução?

Sim, houve redução, Ana Silva de **36.511 ms** para **0.912 ms** e João Santos de **14.664 ms** para **0.372 ms**

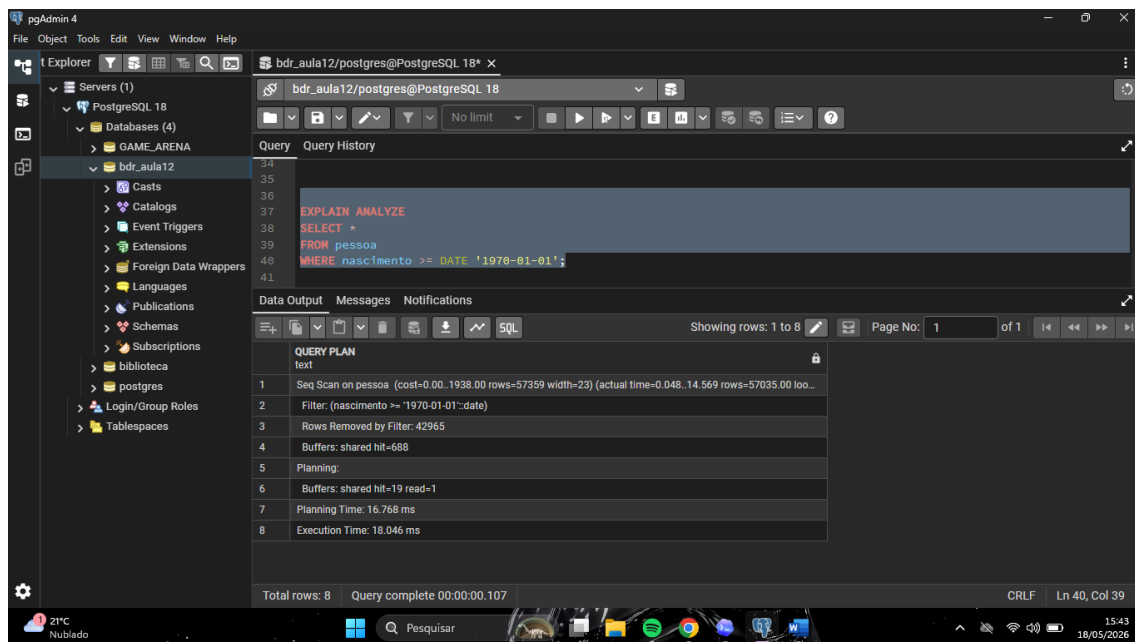
- O índice foi utilizado em ambos os nomes testados? Justifique com base na saída do plano.

Sim, em ambos os planos aparece explicitamente **Bitmap Index Scan on idx_pessoa_nome**, mostrando que o índice foi usado nas duas consultas

EX02.A)



C)



D)

- O índice criado foi utilizado pelo PostgreSQL?

Não

- O plano de execução mudou após a criação do índice?

Não

- Houve melhora significativa no tempo de execução?

Não, os resultados foi de 11.173 ms para 18.046 ms

- Por que o otimizador pode optar por não utilizar um índice, mesmo quando ele existe?

Porque a condição nascimento >= '1970-01-01' retorna muitos registros (baixa seletividade, ~57 mil linhas), e nesse cenário ler grande parte da tabela em sequência costuma ser mais barato do que acessar índice + tabela.

EX03. A)

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show the 'bdr_aula12' database. The main window displays a SQL query in the 'Query' tab:

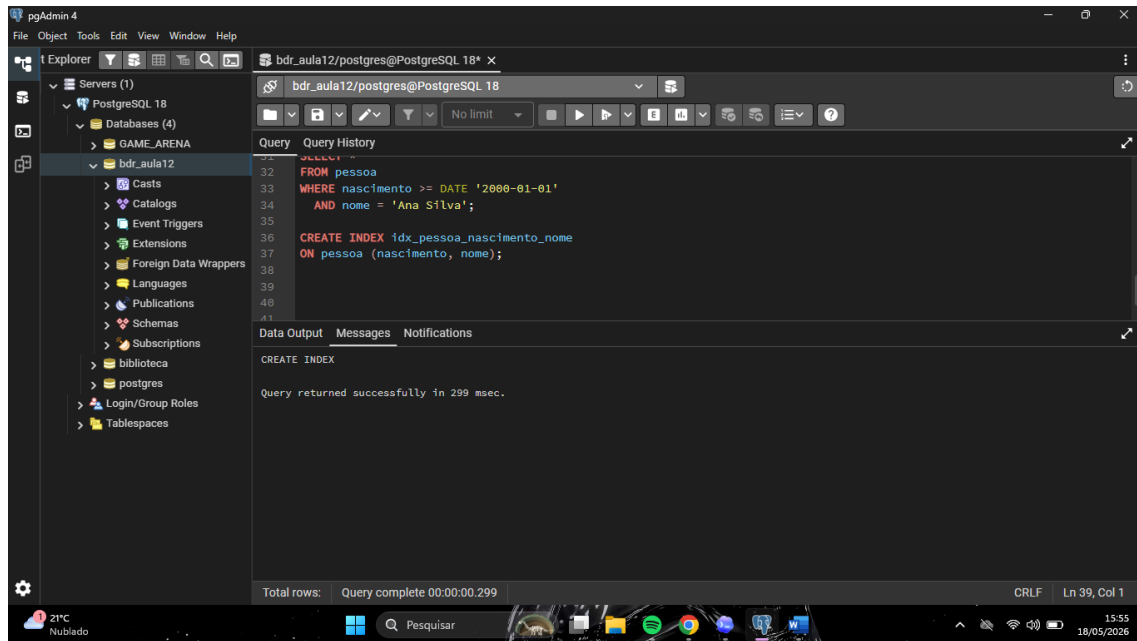
```
28  
29  
30 EXPLAIN ANALYZE  
31 SELECT *  
32 FROM pessoa  
33 WHERE nascimento >= DATE '2000-01-01'  
34 AND nome = 'Ana Silva';  
35  
36  
37
```

Below the query, the 'Data Output' tab shows the 'QUERY PLAN' for the executed query:

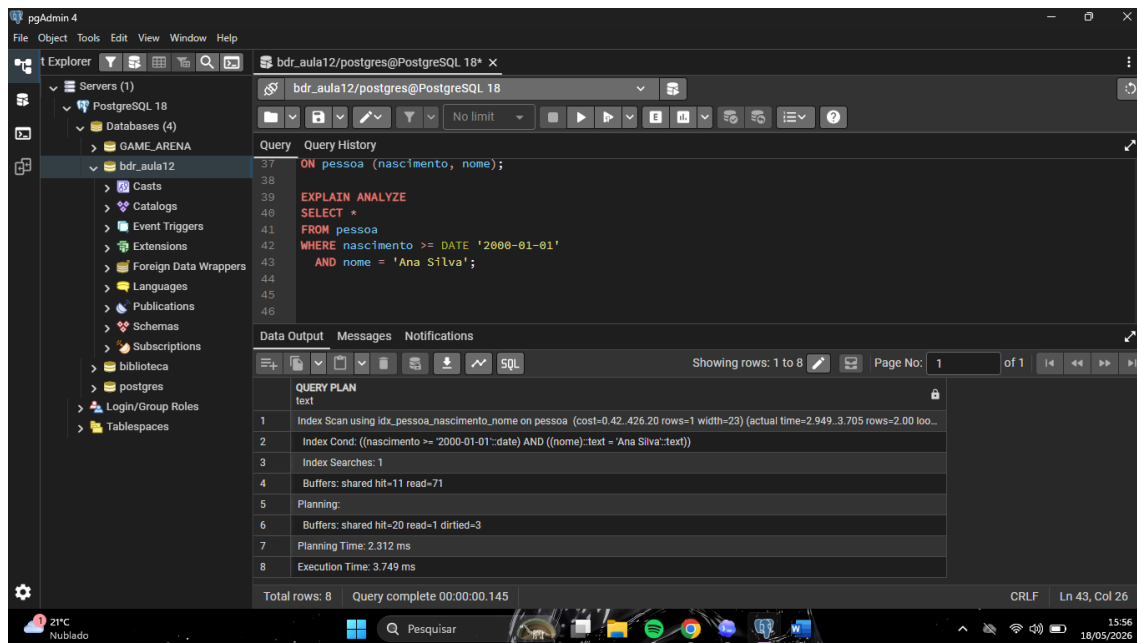
Step	Operation	Cost	Time	Rows	Width	Actual Time	Actual Rows	Actual Width
1	Seq Scan on pessoa	(cost=0.00..2188.00 rows=1 width=23)	(actual time=6.407..10.053 rows=2.00 loop=1)	2	23	6.407	2	23
2	Filter: ((nascimento >= '2000-01-01'::date) AND ((nome)::text = 'Ana Silva'::text))			99998				
3	Rows Removed by Filter: 99998							
4	Buffers: shared hit=688							
5	Planning:							
6	Buffers: shared hit=6 dirtied=1							
7	Planning Time: 1.342 ms							
8	Execution Time: 10.114 ms							

At the bottom, the status bar indicates 'Total rows: 8' and 'Query complete 00:00:00.101'.

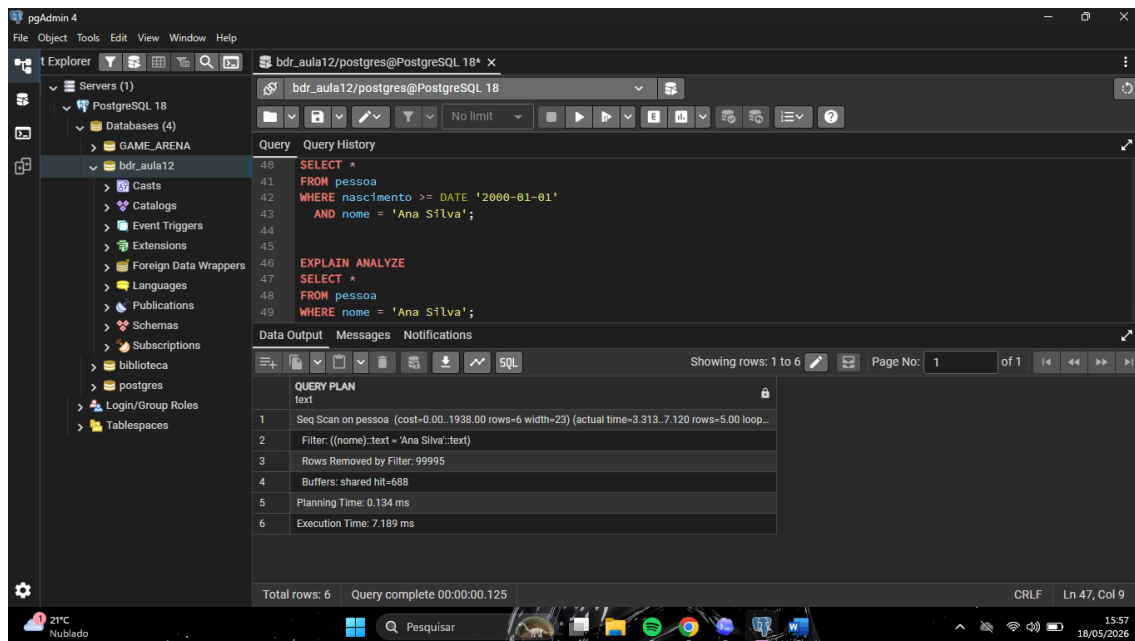
B)



C)



D)



E)

- Qual plano foi utilizado antes da criação do índice composto?

Seq Scan

- Qual plano foi utilizado após a criação do índice composto?

Index Scan

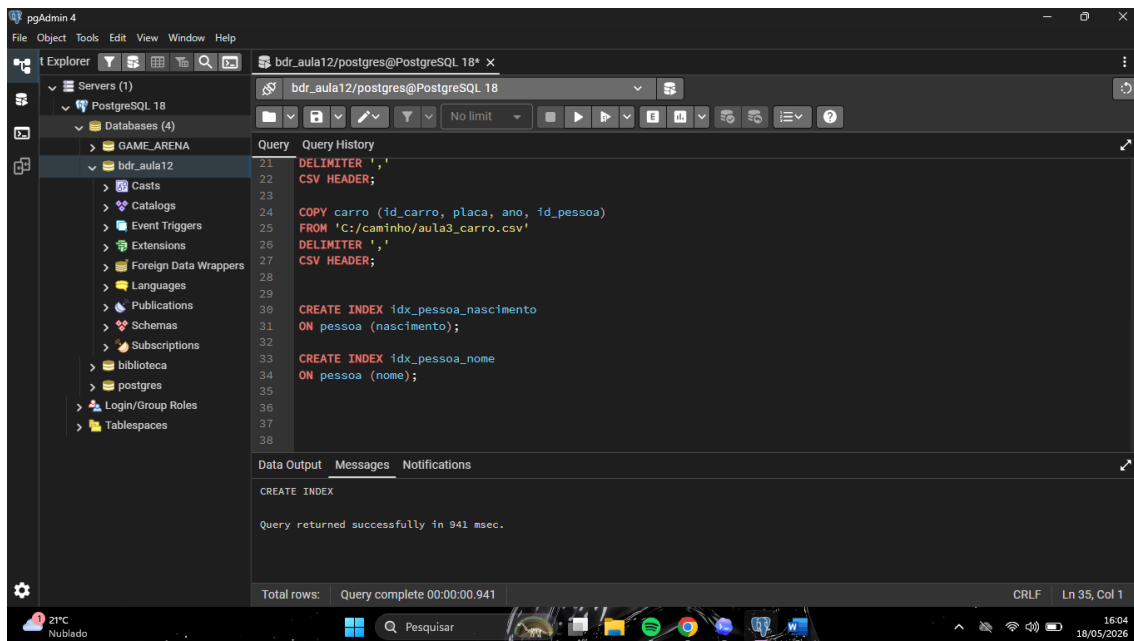
- O índice composto foi utilizado na consulta que filtra apenas por nome?

Não

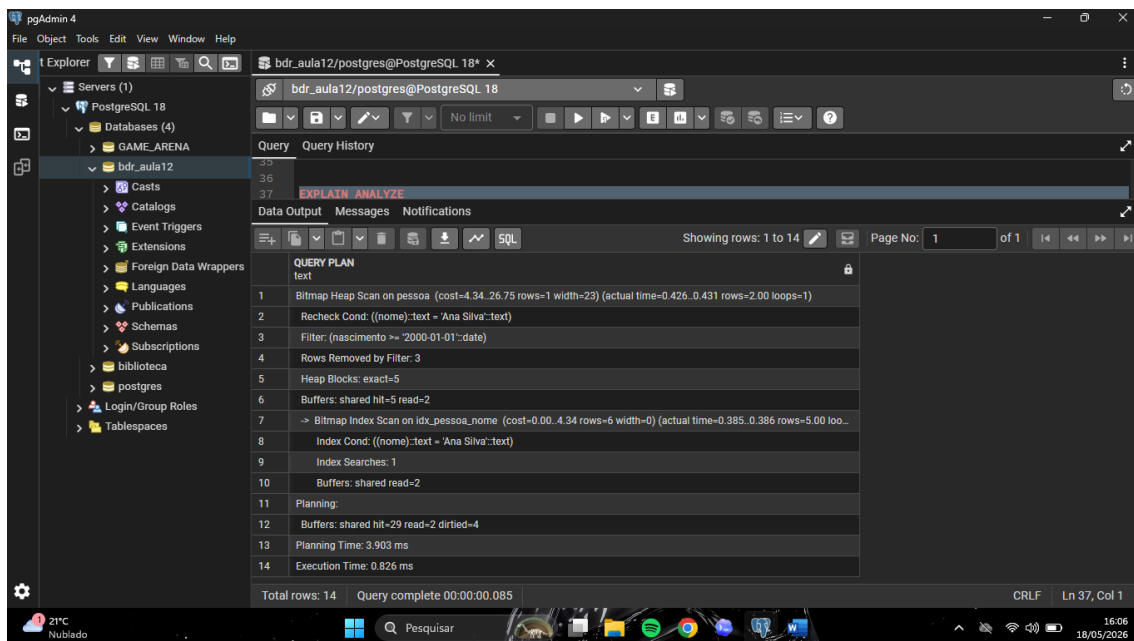
- Por que a ordem das colunas no índice composto é relevante?

Porque o PostgreSQL aproveita o índice composto a partir da coluna da esquerda para a direita

EX04.A)



B)



C)

- O PostgreSQL utilizou os dois índices simples na consulta?

Não, usou apenas o índice nome

- Qual é o papel do operador BitmapAnd no plano de execução?

Serve para combinar os resultados de dois ou mais índices bitmap quando isso é vantajoso.

- O que o plano efetivamente fez?

Buscou primeiro os registros pelo índice de nome e, em seguida, filtrou esses resultados pela condição de nascimento

EX05.

The screenshot shows the pgAdmin 4 interface with a query executed on the 'bdr_aula12' database. The query is:

```
EXPLAIN ANALYZE
SELECT *
FROM carro
WHERE ano BETWEEN 2015 AND 2020;
```

The query plan output shows a sequential scan on the 'carro' table:

Step	Operation	Cost	Time	Rows	Width	Actual Time	Actual Rows	Actual Width
1	Seq Scan on carro	(cost=0.00..2137.00 rows=17357 width=20)	(actual time=0.922..43.904 rows=17016 loops=1)					
2	Filter: ((ano >= 2015) AND (ano <= 2020))							
3	Rows Removed by Filter: 82984							
4	Buffers: shared hit=637							
5	Planning:							
6	Buffers: shared hit=9 dirtied=2							
7	Planning Time: 5.339 ms							
8	Execution Time: 45.994 ms							

Total rows: 8. Query complete 00:00:00.133

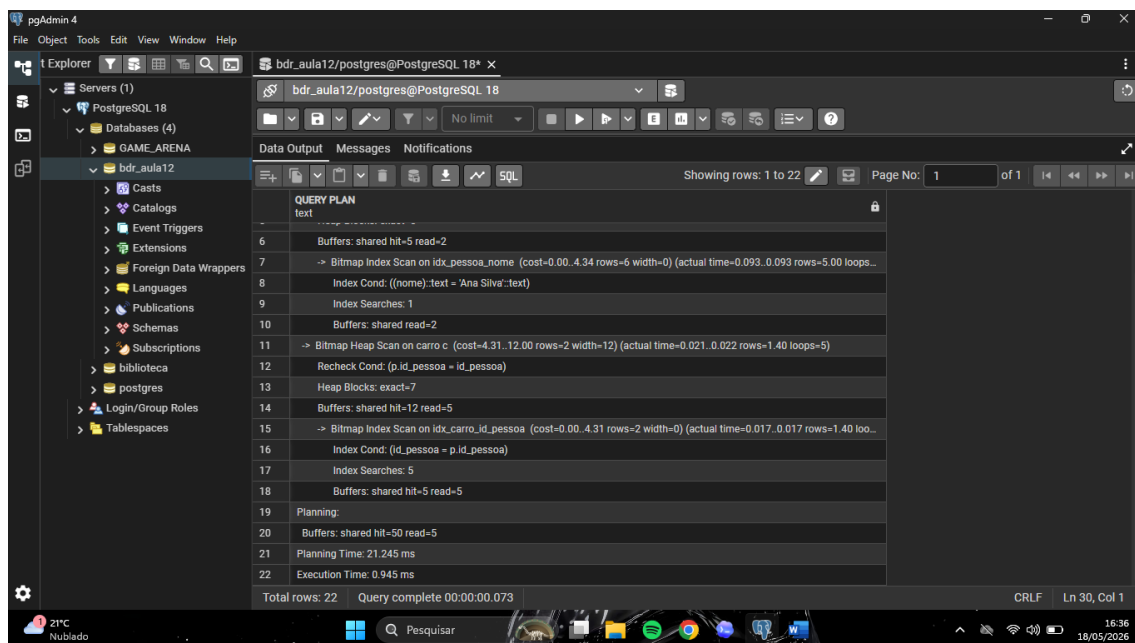
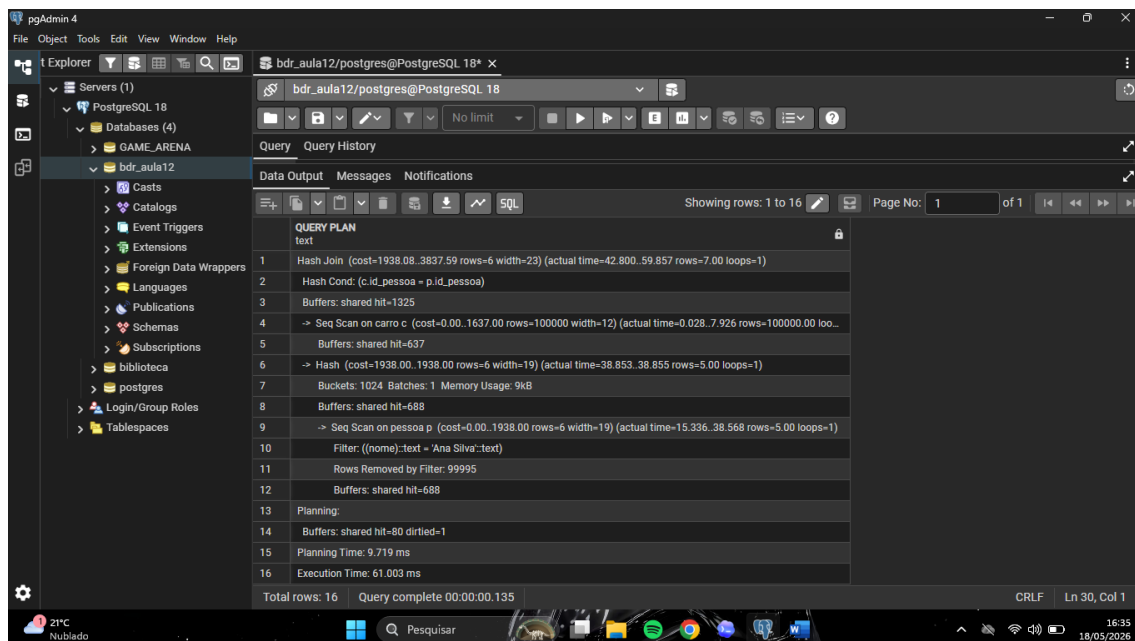
The screenshot shows the pgAdmin 4 interface with the same query executed. The query plan output shows a bitmap index scan on the 'carro' table:

Step	Operation	Cost	Time	Rows	Width	Actual Time	Actual Rows	Actual Width
1	Bitmap Heap Scan on carro	(cost=242.20..1139.56 rows=17357 width=20)	(actual time=2.561..5.643 rows=17016 loops=1)					
2	Recheck Cond: ((ano >= 2015) AND (ano <= 2020))							
3	Heap Blocks: exact=637							
4	Buffers: shared hit=637 read=17							
5	Bitmap Index Scan on idx_carro_ano	(cost=0.00..237.86 rows=17357 width=0)	(actual time=2.451..2.451 rows=17016 loops=1)					
6	Index Cond: ((ano >= 2015) AND (ano <= 2020))							
7	Index Searches: 1							
8	Buffers: shared read=17							
9	Planning:							
10	Buffers: shared hit=22 read=1							
11	Planning Time: 18.305 ms							
12	Execution Time: 6.933 ms							

Total rows: 12. Query complete 00:00:00.074

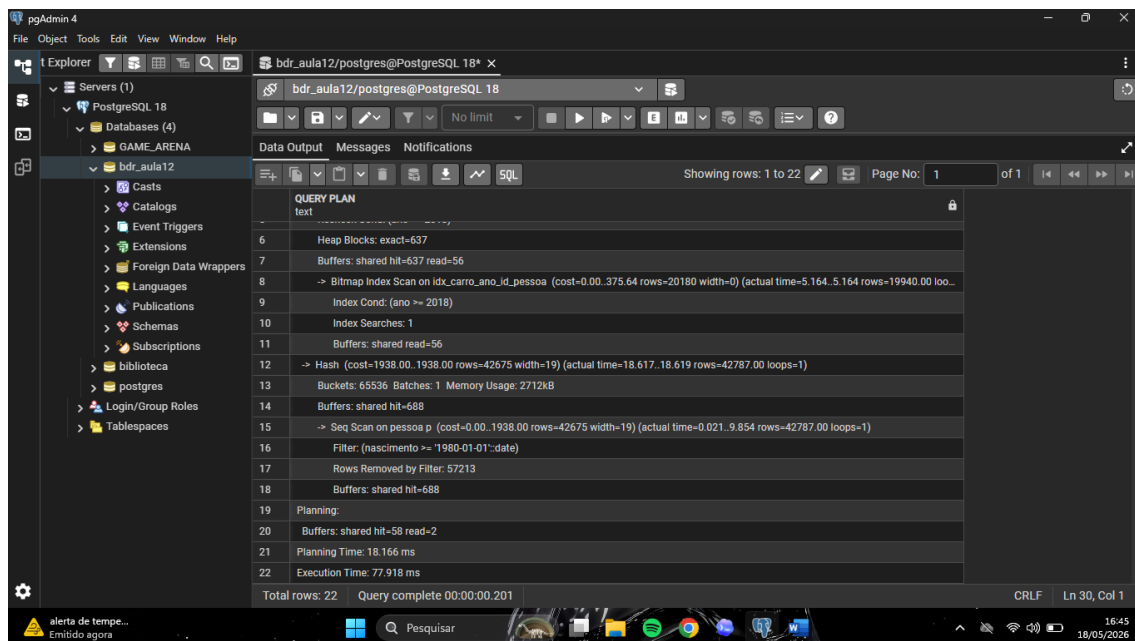
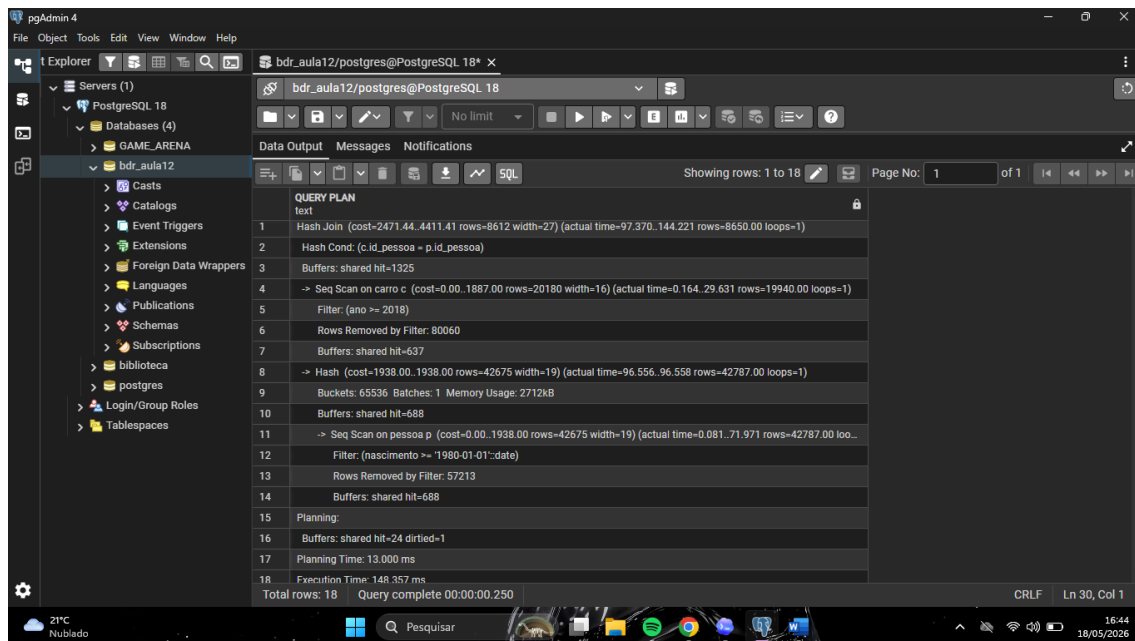
Houve melhora significativa de desempenho, queda grande no tempo, mostrando que o índice em ano é adequado para filtro por intervalo

EX06.



A criação de índice na coluna de filtro (p.nome) e na chave estrangeira usada no JOIN (c.id_pessoa) reduziu drasticamente o custo da consulta, mostrando a importância de indexar colunas de filtro e junção

EX07.



Houve melhora de desempenho, principalmente no filtro da tabela carro, mas pessoa ainda ficou com Seq Scan, então a otimização foi parcial

EX08.A)

pgAdmin 4

File Object Tools Edit View Window Help

Server Explorer: Servers (1) > PostgreSQL 18 > Databases (4) > GAME_ARENA > bdr_aula12

Query Editor: bdr_aula12/postgres@PostgreSQL 18

```
EXPLAIN ANALYZE
SELECT *
FROM pessoa
WHERE nascimento BETWEEN DATE '1980-01-01' AND DATE '1990-12-31';
```

Query History

Data Output: Messages Notifications

Showing rows: 1 to 8

Page No: 1 of 1

QUERY PLAN

1 Seq Scan on pessoa (cost=0.00..2188.00 rows=15794 width=23) (actual time=0.434..19.722 rows=15745.00 loo...

2 Filter: ((nascimento >= '1980-01-01'::date) AND (nascimento <= '1990-12-31'::date))

3 Rows Removed by Filter: 84255

4 Buffers: shared hit=688

5 Planning:

6 Buffers: shared hit=9 dirtied=1

7 Planning Time: 10.131 ms

8 Execution Time: 22.524 ms

Total rows: 8 Query complete 00:00:00.108

CRLF Ln 35, Col 1

20°C Nublado

Pesquisar

17:15 18/05/2026

B)

pgAdmin 4

File Object Tools Edit View Window Help

Server Explorer: Servers (1) > PostgreSQL 18 > Databases (4) > GAME_ARENA > bdr_aula12

Query Editor: bdr_aula12/postgres@PostgreSQL 18

```
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'pessoa';
```

Query History

Data Output: Messages Notifications

Showing rows: 1 to 2

Page No: 1 of 1

indexname	indexdef
pessoa_pkey	CREATE UNIQUE INDEX pessoa_pkey ON public.pessoa USING btree (id_pessoa)
idx_pessoa_nascimento_g...	CREATE INDEX idx_pessoa_nascimento_gist ON public.pessoa USING gist (nascime...

Total rows: 2 Query complete 00:00:00.391

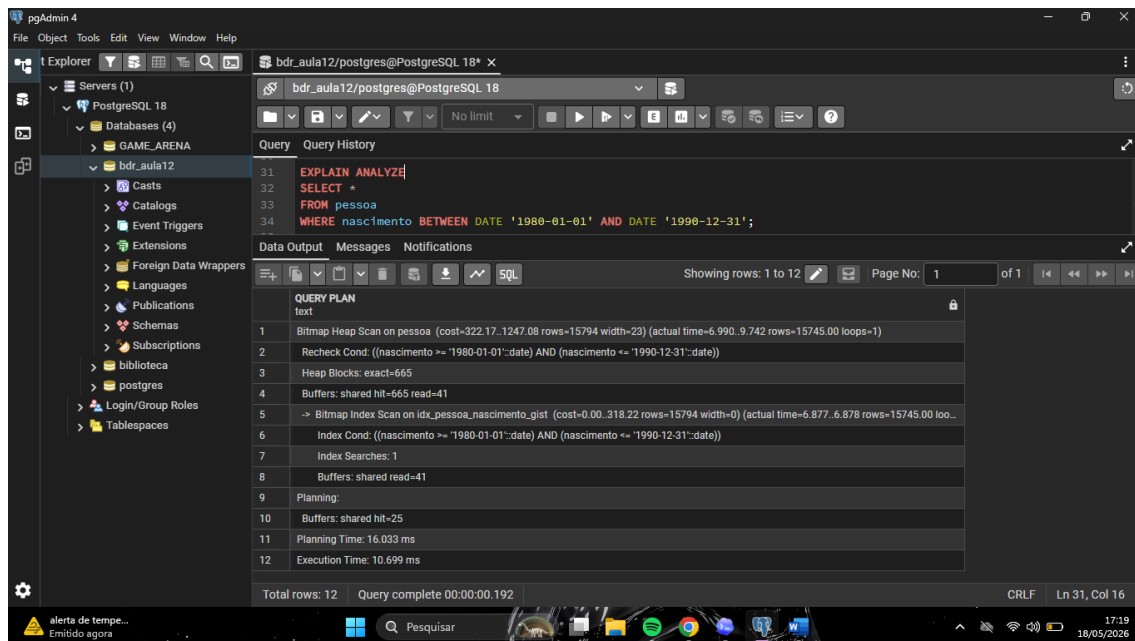
CRLF Ln 42, Col 16

alerta de tempe... Emitido agora

Pesquisar

17:18 18/05/2026

C)



D)

- Qual plano foi utilizado antes da criação do índice GiST?

Seq Scan

- O índice GiST foi utilizado após a criação?

Sim

- Houve melhora no tempo de execução?

Sim, de 22.524 ms para 10.699 ms